

학습 노트 (2). 바이브 코딩으로 만든 우주 침략자, 그리고 손으로 만든 신경망

공과대학 기계공학부 안준영 학번 2024-11078 2026년 9월 24일

들어가며

이번 주에는 두 가지 일을 했다. 하나는 교수님 블로그의 바이브 코딩으로 우주 침략자 게임 만들기 시리즈에 부과된 과제 두 개를 수행한 것이다. Q.09e와 이전 단계의 코드를 diff로 비교하는 과제, 그리고 고장 난 Makefile.mk을 고쳐서 빌드와 실행을 성공시키는 과제다. 다른 하나는 수업에서 언급된 책 Make Your Own Neural Network를 읽고, 책 코드를 그대로 돌리는 대신 내용을 이해한 뒤 직접 신경망을 학습시켜 본 것이다. 이 기록을 과제 보고서 뒤에 함께 실는다.

과제 1. diff 비교

지시대로 Q.09e 디렉토리에서 diff를 실행했다.

```
diff -y -w 128 ../Q.09c_Enhance_Keyboard_Stim/sc_invaders_TB.cpp \  
./sc_invaders_TB.cpp | less
```

sc_glcd128x64_TLM.cpp도 Q.09d 버전과 같은 방법으로 비교했다. 결과가 조금 의외였다. 주석을 걷어내고 코드 로직만 놓고 볼 때, 기능이 바뀐 것이 하나도 없었다. sc_invaders_TB.cpp에서는 주석 문구 다듬기와 로그 메시지 한 줄이 바뀌었고("시뮬레이터 조작을 시작합니다"가 "에뮬레이터 키보드 제어권을 활성화합니다"로), sc_glcd128x64_TLM.cpp도 주석 보강과 에러 메시지 문구, for(int y=0을 for(int y = 0으로 고치는 정도의 스타일 정리가 전부였다.

진짜 기능 변화는 Q.09c와 Q.09d에서 이미 다 일어났던 것이다. Q.09c에서 키보드 자극 주입이 들어갔고, Q.09d에서 SDL 이벤트 루프가 보완되면서 창 프리징이 해결되고 SDL_QUIT 시 Verilated::gotFinish(true)와 sc_stop()이 연동됐다. Q.09e는 그 결과를 제출 가능한 상태로 정리한 버전이다. diff 하나로 바이브 코딩의 실제 작업 방식이 보였다. AI가 기능을 만들어 내는 단계가 있고, 그 다음에 그것을 검토하고 다듬어 완성형으로 정리하는 단계가 따로 있다. 나는 이 뒤쪽 단계에서 엔지니어링이 일어난다고 생각한다. 코드가 돌아가는 것과 코드가 맞는 것은 다른 일이고, 그 차이는 diff를 읽는 사람이 직접 좁혀야 한다.

과제 2. Makefile.mk 고치기

Q.09e_SystemC_Env_Quiz에서 make -f Makefile.mk을 실행하면 다음 오류가 나면서 바로 멈춘다.

```
%Error: Cannot find file containing module: space_invaders_engine_glcd.v
```



```

junyoung2@junyoung2:~/ETRI050_DesignKit/Projects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_Quiz$ grep -n "unsigned int" sc_main.cpp sc_glcd128x64_TLM.h sc_glcd128x64_TLM.cpp
sc_main.cpp:23:     sc_signal<unsigned int> sig_lcd_x;
sc_main.cpp:24:     sc_signal<unsigned int> sig_lcd_y;
sc_glcd128x64_TLM.h:16:     sc_in<unsigned int> lcd_x;
sc_glcd128x64_TLM.h:17:     sc_in<unsigned int> lcd_y;
sc_glcd128x64_TLM.cpp:72:         unsigned int x = lcd_x.read();
sc_glcd128x64_TLM.cpp:73:         unsigned int y = lcd_y.read();

```

그림 3. 수정 후 unsigned int로 타입 통일 확인

네 군데를 고치고 나니 빌드와 실행에 성공했다.

```

junyoung2@junyoung2: ~/ETRI050_DesignKit/Projects/MyChi...
no-bool-operation -Wno-sign-compare -Wno-uninitialized -Wno-unused-but-set-variable -Wno-unused-parameter -Wno-unused-variable -Wno-shadow -std=c++17 -I/usr/include/SDL2 -D_REENTRANT -I/opt/systemc/include -Os -c -o sc_glcd128x64_TLM.o ./sc_glcd128x64_TLM.cpp
g++ -I. -MMD -I/usr/share/verilator/include -I/usr/share/verilator/include/vltstd -DVM_COVERAGE=0 -DVM_SC=1 -DVM_TRACE=0 -faligned-new -fcf-protection=none -Wno-bool-operation -Wno-sign-compare -Wno-uninitialized -Wno-unused-but-set-variable -Wno-unused-parameter -Wno-unused-variable -Wno-shadow -std=c++17 -I/usr/include/SDL2 -D_REENTRANT -I/opt/systemc/include -Os -c -o sc_main.o ./sc_main.cpp
g++ sc_glcd128x64_TLM.o sc_invaders_TB.o sc_main.o verilated.o Vspace_invaders_engine_glcd_ALL.a -lSDL2 -L/opt/systemc/lib -lsystemc -lsystemc -o Vspace_invaders_engine_glcd
make[1]: Leaving directory '/home/junyoung2/Desktop/ETRI-0.5um-CMOS-MPW-Std-Cell-DK/Projects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_Quiz/obj_dir'

=====
[SUCCESS] 아케이드 빌드가 성공적으로 완수되었습니다!
-> 실행 바이너리 실행 지점 명령: './obj_dir/Vspace_invaders_engine'
=====

```

그림 4. 빌드 성공

```

junyoung2@junyoung2:~/ETRI050_DesignKit/Projects/MyChip_Games/space_invader_by_Gemini/Q.09e_SystemC_Env_Quiz$ ./obj_dir/Vspace_invaders_engine_glcd

SystemC 3.0.2-Accellera --- Sep 16 2026 12:18:17
Copyright (c) 1996-2025 by all Contributors,
ALL RIGHTS RESERVED
[SystemC TB] 하드웨어 전원 리셋 해제 완료. 에뮬레이터 키보드 제어권을 활성화합니다.

```

그림 5. 실행 성공, SystemC 3.0.2 배너와 키보드 제어권 활성화

돌아볼 때 네 가지는 모두 코드의 논리가 아니라 환경의 문제였다. 어느 풀팜에 무엇이 있는지, 라이브러리가 어디 설치됐는지, 도구가 무엇을 생성하는지, 버전마다 무엇이 다른지. 이런 것들은 AI가 대신 파악하기 어렵다. 내 컴퓨터의 상태를 아는 사람은 나뿐이기 때문이다.

실행 확인

실행하니 게임 화면이 아니라 체크무늬가 가득 찬 창이 떠서 처음에는 고장 난 줄 알았다. 알고 보니 엔진이 시작 대기 상태에서 출력하는 테스트 패턴이었다. scan_x와 scan_y를 XOR한 값을 픽셀에 찍는 것인데, 베릴로그 코드에서 그 계산식을 그대로 찾을 수 있었다. 화면에 아무것도 안 뜬 것이 아니라, 회로가 매 픽셀 좌표를 계산해서 찍고 있었던 것이다.

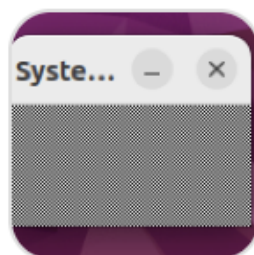


그림 6. START 대기 상태의 XOR 체크무늬

스페이스바를 누르자 5×5 외계인 대형이 나타났고 아래에는 패들이 놓였다. 지난주 탁구 게임기에서는 ATmega328p의 소프트웨어가 화면을 그렸는데, 이번에는 베릴로그 회로가 그린다. 키를 누르는 순간의 입력이 SystemC 테스트벤치를 거쳐 회로의 클럭으로 들어가는 길을 직접 따라가 봤더니, 같은 게임 화면도 다르게 보였다.

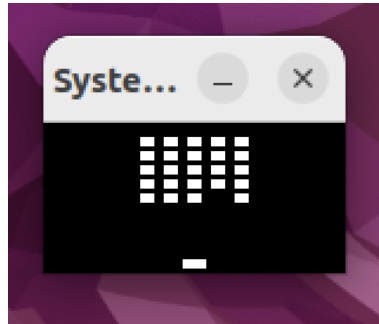


그림 7. 게임 시작, 5×5 외계인 대형과 패들

과제를 마치고

이번 주 메일의 토론 주제가 “느낌만으로 설계했다고 할 수 있는가”였다. 나는 이렇게 답하고 싶다. AI가 95%를 커버하는 시대에도 나머지 5%에서는 인간의 개입이 필수적이고, 그 5%가 실제 제품과 시제품을 가른다.

이번 과제가 그 실증이었다. Gemini는 Space Invaders의 RTL을 만들어 냈고 Q.01부터 Q.12까지의 기능 구현 대부분이 AI의 몫이었다. 그런데 그 결과물을 내 컴퓨터에서 돌리는 마지막 구간은 사람의 몫으로 남았다. 빠진 파일의 출처를 추적하고, 라이브러리 경로를 잡고, 도구 버전이 바뀐 인터페이스를 읽어 내는 일이다. 코드는 완성되어 있었지만 내 환경과의 연결은 끊겨 있었고, 그 연결을 잇는 데 필요한 것은 코드 생성 능력이 아니라 시스템 전체를 꿰뚫어 보는 혜안이었다.

과제 1의 diff가 보여준 것도 같은 그림이다. 기능은 Q.09c와 Q.09d에서 만들어졌지만, Q.09e라는 완성형은 누군가가 전체를 검토하고 정리해야 나온다. 바이브 코딩이 아무리 좋아져도 검토와 정리, 검증의 단계에서는 엔지니어링이 일어난다. 그래서 바이브 코딩 시대에 엔지니어의 역할은 사라지는 것이 아니라 이동한다고 본다. 코드를 쓰는 사람에서, AI가 만든 것을 읽고 검증하고 책임지는 사람으로. 앞으로의 설계자에게 필요한 것은 타이핑 속도가 아니라 시스템 전체를 한눈에 담는 이해일 것이다.

DPI-C 대신 SystemC 테스트벤치로 옮긴 구성에는 동의한다. 자극 주입과 검증 로직이 표준화된 언어 안에 들어오면 회귀 테스트와 자동화로 가져가기 쉬워지고, 테스트 환경이 그 자체로 검토 가능한 문서가 되기 때문이다. 다만 이번 실습에서 진짜로 발목을 잡은 것은 언어 선택이 아니라 도구 체인의 버전이었다. 어느 경로를 택하든 그 밑에 깔린 도구를 이해하는 일은 피할 수 없다.

Make Your Own Neural Network

1부. 책을 읽고

Tariq Rashid의 Make Your Own Neural Network를 읽었다. 신경망을 수학 기호부터 세우는 책이 아니라, 작은 코드 조각들을 쌓아가면서 보여주는 책이다. 읽으면서 남은 생각 다섯 가지를 적는다.

첫째, 학습을 오차를 고쳐 나가는 과정으로 설명한 부분이 좋았다. 출력과 정답의 차이를 거슬러 올라가면서 각 가중치가 오차에 기여한 만큼 나눠 갖게 하는 방식이다. 회로를 배우는 눈으로 볼 때 오류가 흐르는 경로를 배선하는 일처럼 읽혔다. 둘째, 분류 예제를 따라가면서 학습률을 직접

바꿔 보게 한 구성이 좋았다. 너무 크면 발산하고 너무 작으면 더딘 것이 바로 보였다. 셋째, 뉴런 모델에 활성화 함수가 들어와 비선형성을 더하면서 모델의 학습성이 커지는 부분이 인상적이었다. 시그모이드가 단순한 완충이 아니라 표현력을 만드는 장치라는 것을 그림으로 이해했다. 넷째, 학습률에 스위트 스팟이 있다는 이야기가 책 전반에서 반복됐다. 한 번의 우연이 아니라 구조적인 현상이라는 뜻이다. 다섯째, 신경망의 종류에 따라 데이터 사이즈의 효과성도 달라진다는 관찰이 있었다. 같은 데이터도 네트워크가 다르며 다르게 소화한다는 것이다.

이 다섯 가지를 그냥 읽고 넘기지 않고 직접 확인해 보기로 했다. 책의 코드를 그대로 돌리는 것이 아니라, 이해한 내용을 바탕으로 간단한 신경망 학습을 돌려 보는 것이 목표였다.

2부. 직접 돌려 본 신경망

3층 신경망(입력 784, 은닉, 출력 10)을 MNIST 손글씨 숫자 데이터로 학습시켰다. 먼저 학습률 스윙부터 했다. 은닉 노드 100 개, 1 에폭 조건에서 학습률만 바꿔 가며 성능을 잰다.

```
lr = 0.01 -> 92.12%
lr = 0.1  -> 95.10%
lr = 0.2  -> 95.34%    <- sweet spot
lr = 0.3  -> 94.72%
lr = 0.6  -> 90.22%
```

책이 보고한 양상과 거의 같게 나왔다. 0.01에서는 덜 배우고, 0.6에서는 배운 것을 망치고, 그 사이 0.2 부근에 스위트 스팟이 있다. 책을 읽을 때는 그럴겠거니 했던 곡선인데, 직접 찍은 숫자로 똑같이 나오니까 느낌이 달랐다.

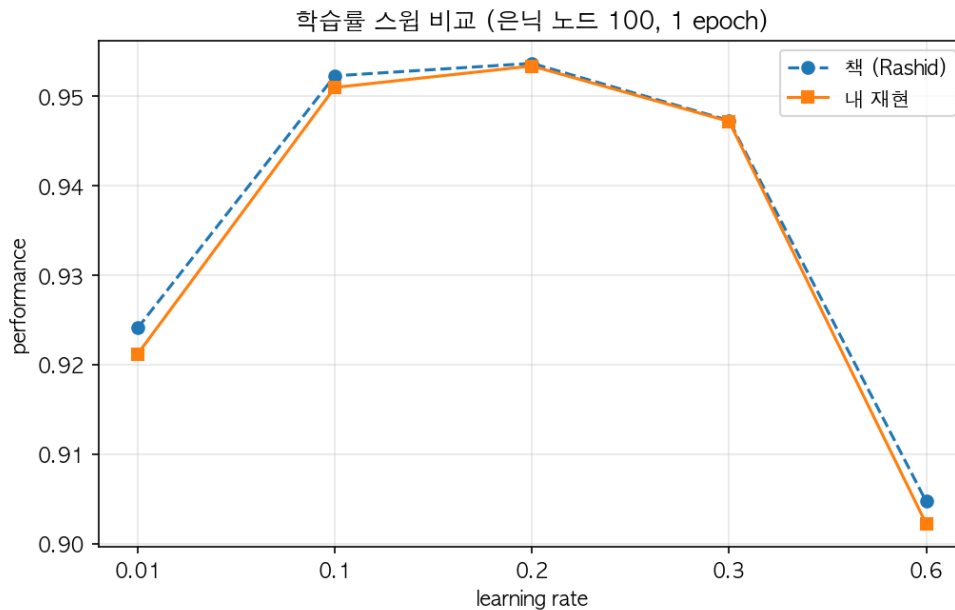


그림 8. 학습률 스윙 결과, 책의 양상과 비교

이어서 최종 설정(은닉 노드 200 개, 학습률 0.1, 5 에폭)으로 돌렸더니 97.52%가 나왔다. 책의 기록은 97.51%다. 노트북에서 26 초 걸린 학습이었다. 역방향 질의도 따라해 봤다. 출력을 특정 숫자로 고정하고 신호를 거꾸로 흘려서, 네트워크가 그리는 숫자의 상을 얻는 실험이다. 분류만 하던 신경망이 사실은 안쪽에 숫자의 이미지를 품고 있다는 것을 눈으로 보여줘서, 이 책에서 가장 재미있는 뒤집기였다.

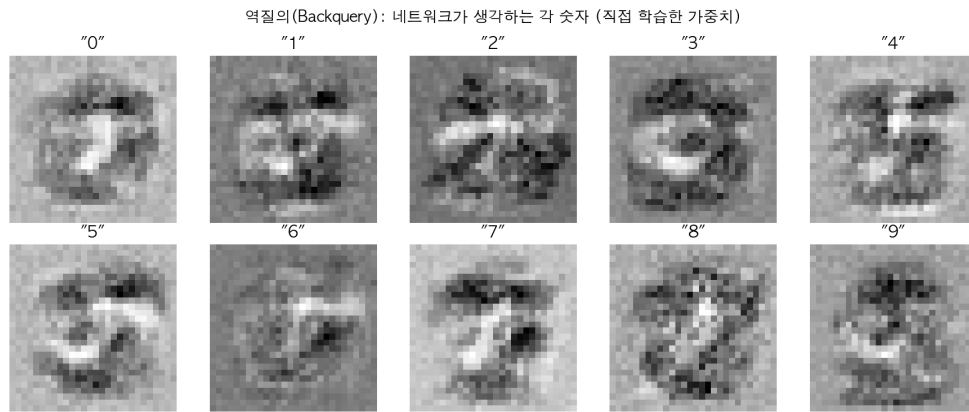


그림 9. 역방향 질의로 얻은 각 숫자의 상

책 카피

읽은 판본의 목차를 그대로 싣는다. Part 1(How They Work)에서 개념을 쌓고 Part 2(DIY with Python)에서 손으로 만드는 이중 구조인데, 위의 1부와 2부 기록이 이 순서를 그대로 따라간 것이다.

Contents

[Prologue](#)

[The Search for Intelligent Machines](#)
[A Nature Inspired New Golden Age](#)

[Introduction](#)

[Who is this book for?](#)
[What will we do?](#)
[How will we do it?](#)
[Author's Note](#)

[Part 1 - How They Work](#)

[Easy for Me, Hard for You](#)
[A Simple Predicting Machine](#)
[Classifying is Not Very Different from Predicting](#)
[Training A Simple Classifier](#)
[Sometimes One Classifier is Not Enough](#)
[Neurons, Nature's Computing Machines](#)
[Following Signals Through A Neural Network](#)
[Matrix Multiplication is Useful... Honest!](#)
[A Three Layer Example with Matrix Multiplication](#)
[Learning Weights From More Than One Node](#)
[Backpropagating Errors From More Output Nodes](#)
[Backpropagating Errors To More Layers](#)
[Backpropagating Errors with Matrix Multiplication](#)
[How Do We Actually Update Weights?](#)
[Weight Update Worked Example](#)
[Preparing Data](#)

[Part 2 - DIY with Python](#)

[Python](#)
[Interactive Python = IPython](#)
[A Very Gentle Start with Python](#)
[Neural Network with Python](#)
[The MNIST Dataset of Handwritten Numbers](#)

그림 10. Make Your Own Neural Network 목차

내 아이디어

이 신경망이 학습과 추론 내내 하는 일은 결국 행렬 곱셈다. 곱하고 더하기의 반복이다. 97.52%라는 숫자는 수백만 번의 MAC(곱셈 누산)이 만든 결과다. 지난주 탁구 게임기 에세이에서 전용 칩이 잘하는 일을 정해진 일의 반복이라고 적었는데, MAC이 정확히 그런 일이다. 노트북에서 26 초 걸린 이 연산이 전용 회로에서는 어떤 모습일지, 이번에는 내가 직접 학습시킨 가중치를 놓고 생각해 볼 수 있었다.

여기에 요즘 내가 만들고 싶은 것을 적어 본다. 나는 지금 지구 시스템의 world model이라는 연구 방향에 빠져 있다. 지구를 위성사진 더미가 아니라 하나의 coupled dynamical physical system으로 보고, 그 상태의 시간 전개를 함수 공간에서 배우는 모델이다. 내 말로 하면 슈퍼컴퓨터 안에 지구라는 모델을 가져와서 키우는 느낌이다. 그 안에서 내가 가장 끌리는 문제는 꼬리다. 확률은 가장 낮지만 임팩트가 엄청 크고 지구사를 바꾸는 사건들이다. 통계적인 방식은 데이터가 없는 그 자리에서 가장 약하다. 그런데 꼬리에서도 물리법칙은 계속 성립한다. 그래서 물리를 아는 모델만이 통계가 멈추는 자리로 계속 걸어 들어갈 수 있다. 이 방향에서 물리는 장식이 아니라 꼬리로 가는 허가증이다.

이 얘기를 하드웨어 수업 보고서에 쓰는 이유가 있다. evolution 연산자가 고전 시뮬레이터보다 수백, 수천 배 빠르게 동역학을 굴릴 수 있다면 꼬리는 비로소 계산 가능해진다. 희귀사건의 확률과 경로를 얻으려면 수만 번의 롤아웃이 필요하고, 그것은 곧 MAC의 홍수다. 이번에 노트북에서 26 초 만에 돌린 작은 신경망이 그 아주 작은 예고편이다. 꼬리를 계산하는 world model은 알고리즘만으로는 존재할 수 없고, 그 연산을 감당할 전용 하드웨어가 뒷받침되어야 실체가 된다. 이 수업에서 칩 설계를 배우는 이유가 여기에 닿아 있다. 언젠가 그런 모델의 연산을 실리콘 위에 직접 펼쳐 보는 것, 그것이 내가 만들고 싶은 것이다.

책 실험은 이 수업의 최종 과제와도 연결된다고 본다. 학기 후반에 내 칩을 설계할 때 설계가 맞는지 어떻게 확인할 것인가는 이번 바이브 코딩 과제와 같은 문제다. 작게 만들고, 수치로 확인하고, 조건을 바꿔 보는 순환은 하드웨어 설계에서도 그대로 통한다. 다만 하드웨어는 한 번 찍히면 되돌릴 수 없으니, 이 순환을 시뮬레이션 안에서 얼마나 성실하게 도느냐가 실력이 될 것이다. 다음 단계로는 내 손글씨 숫자를 직접 넣어 볼 생각이다. 이 네트워크가 정말 숫자를 아는지 스스로 묻는 가장 직접적인 시험이기 때문이다.

맺으며

이번 주는 누군가가 만들어 준 것을 돌려 보는 주간이었다. AI가 만든 게임을 내 환경에서 빌드했고, 책이 만든 신경망을 내 데이터로 재현했다. 두 경험은 같은 교훈으로 모인다. 만들어진 것은 그대로 돌아가지 않고, 돌아가게 만드는 마지막 5%에서 비로소 내 것이 된다는 것. diff를 읽고, 로그를 읽고, 숫자를 찍어 보는 일. 검증하는 사람만이 설계의 주인이 된다는 것을 이번 주에 배웠다.